

Módulo 4: Aprendizaje Automático

Clase 5: Taller Comparativo de Algoritmos de Clasificación

Diplomado en Ciencia de Datos

17 de julio de 2026

Objetivos de la Sesión

- Comprender que no existe un "algoritmo perfecto" (*No Free Lunch Theorem*).
- Estudiar **k-Nearest Neighbors (k-NN)** y el aprendizaje basado en distancia geométrica.
- Entender la lógica de los **Árboles de Decisión** a través de las métricas de partición (Entropía y Gini).
- Introducir **Naive Bayes**, un clasificador puramente probabilístico.
- Aprender a implementar cada modelo usando Scikit-Learn y comparar sus resultados.

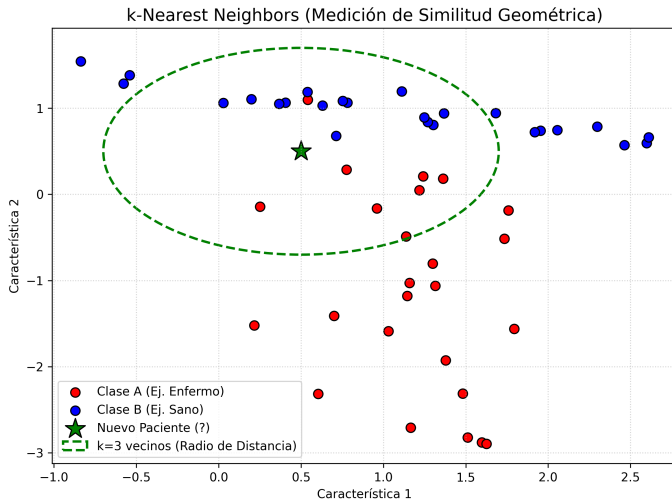
1. k-Nearest Neighbors (k-NN)

¿**Qué es?** Un algoritmo de aprendizaje "perezoso" (*lazy learning*). No construye una ecuación matemática en el entrenamiento; simplemente memoriza la base de datos.

Mecanismo de Decisión:

- Cuando llega un nuevo registro, el algoritmo calcula su **Distancia Euclidiana** hacia todos los puntos existentes.
- Selecciona los k puntos más cercanos (ej. $k = 5$).
- La clase mayoritaria entre esos vecinos determina la predicción.

1. k-Nearest Neighbors (Visualización)



1. k-NN: Implementación y Precauciones

Advertencia Crítica: Debido a que usa distancias geométricas, es obligatorio **Estandarizar (Escalar)** los datos. Si una columna tiene números muy grandes (ej. Sueldo en miles), dominará el cálculo de distancia.

Comandos Scikit-Learn:

```
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
```

```
# Estandarización obligatoria
scaler = StandardScaler()
X_train_esc = scaler.fit_transform(X_train)
```

```
# Instanciación y Entrenamiento
modelo_knn = KNeighborsClassifier(n_neighbors=5)
modelo_knn.fit(X_train_esc, y_train)
```

2. Árboles de Decisión

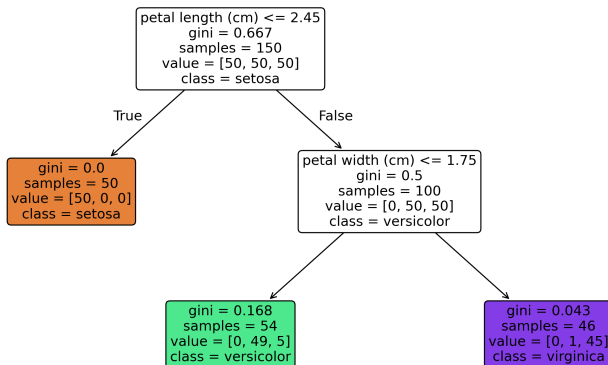
¿**Qué es?** Un modelo basado en reglas lógicas condicionales (*If-Then*). Es altamente intuitivo y fácil de explicar a perfiles de negocio.

Mecanismo de Partición (Splitting):

- **Entropía:** Mide el nivel de desorden o impureza en un nodo.
- **Ganancia de Información:** El árbol evaluará todas las variables y cortará los datos usando la pregunta que maximice la ganancia de información (es decir, la pregunta que separe mejor las clases y reduzca la entropía).
- También se suele utilizar el Índice **Gini** como alternativa a la Entropía por ser computacionalmente más rápido.

2. Árboles de Decisión (Visualización)

Árbol de Decisión: Partiendo el espacio de características (Entropía/Gini)



2. Árboles: Implementación y Precauciones

El Peligro del Sobreajuste (Overfitting): Si no limitamos su crecimiento, el árbol seguirá haciendo preguntas hasta tener una rama para cada registro individual, memorizando los datos pero perdiendo capacidad de generalizar con datos futuros.

Comandos Scikit-Learn:

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import plot_tree # Para graficar el árbol

# Instanciación (Es vital limitar la profundidad max_depth)
modelo_arbol = DecisionTreeClassifier(max_depth=4,
                                      criterion='entropy',
                                      random_state=42)

# Entrenamiento (No requiere escalado previo)
modelo_arbol.fit(X_train, y_train)
```

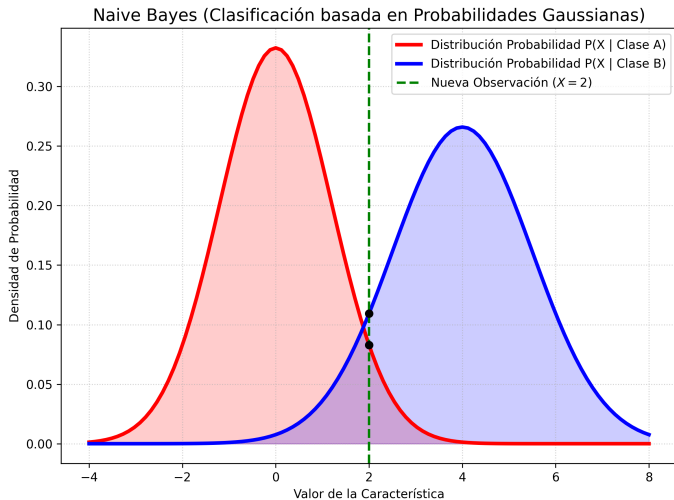

3. Naive Bayes

¿**Qué es?** Un modelo de clasificación basado en el Teorema de Bayes sobre probabilidad condicional.

¿**Por qué "Ingenuo" (Naive)?**

- Asume la independencia condicional fuerte. Supone que la presencia de una característica en particular no está relacionada con la presencia de ninguna otra característica.
- En la realidad, las variables suelen estar correlacionadas, pero hacer esta asunción "ingenua" simplifica drásticamente el cálculo matemático.
- **Ventajas:** Es extremadamente veloz, ideal para datasets con muchísimas dimensiones (ej. Análisis de texto o detección de Spam).

3. Naive Bayes (Visualización Probabilística)



3. Naive Bayes: Implementación

Existen varias implementaciones (Multinomial para conteo de palabras, Bernoulli para binarios). Para variables continuas del mundo físico usamos **Gaussian Naive Bayes**.

Comandos Scikit-Learn:

```
from sklearn.naive_bayes import GaussianNB

# Instanciación
modelo_bayes = GaussianNB()

# Entrenamiento
modelo_bayes.fit(X_train, y_train)

# Predicción
predicciones = modelo_bayes.predict(X_test)
```

Torneo de Clasificadores

Algoritmo	Ventaja	Desventaja	¿Escalar?
k-NN	Excelente con fronteras no lineales	Sufre con muchas variables (Lento)	Sí (Obligatorio)
Decision Tree	Muy interpretable (Visual)	Alta tendencia al Sobreajuste	No
Naive Bayes	Ultra veloz, genial para texto	Pierde precisión si hay correlación	No

Fin de la Presentación Teórica

Transición al Laboratorio Práctico Comparativo:

`05_Comparacion_Clasificadores.ipynb`

`05_Laboratorio_Evaluacion.ipynb`